



1. Resumen Ejecutivo

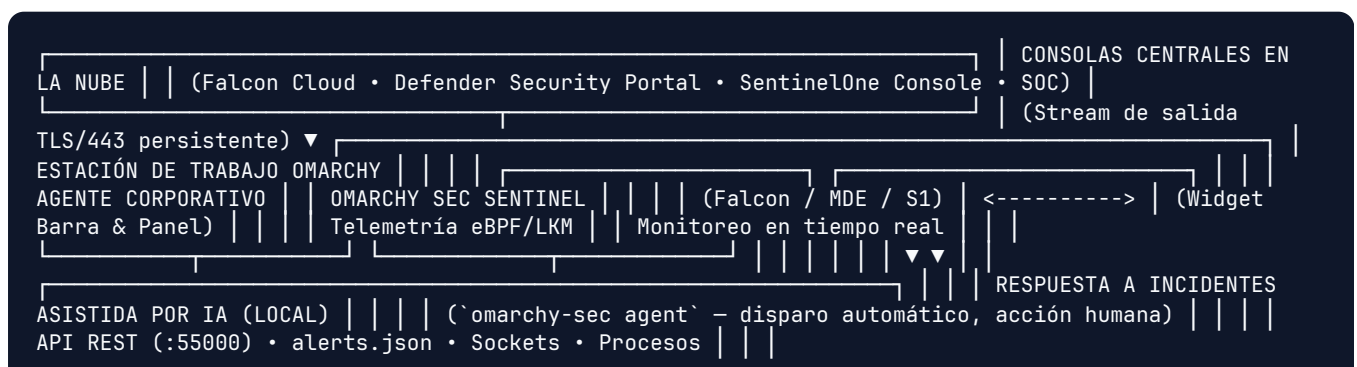
Omarchy Sec es una integración de seguridad de endpoint, agnóstica de proveedor, para estaciones de trabajo **Omarchy Linux (Arch Linux + Hyprland)**. Es un proyecto open source personal, sin auditoría de seguridad externa.

Consta de un motor de detección que reporta qué sensor EDR está activo (**CrowdStrike Falcon, Microsoft Defender, SentinelOne, Cortex XDR, Wazuh, Falco/Tetragon, auditd**), un cliente REST para la API de Wazuh, un stack single-node de Wazuh en Docker, un widget de barra para Quickshell y un watcher que abre el agente de código con el contexto del incidente cargado. Documenta además los procedimientos de conversión de paquetes (debtpap, rpmextract) que hacen instalable un sensor corporativo en una distribución rolling-release.

Pilares Clave:

- **Agnóstico & Multi-Sensor:** un único comando (`omarchy-sec-detect`) reporta en JSON qué sensores están activos o inactivos, consultando units de systemd, procesos y contenedores. No extrae salud ni versión del motor del vendor: eso no se obtiene de un chequeo de estado de unit.
- **Respuesta a incidentes asistida por IA ("Call Agent"):** el disparo es automático (alerta de nivel ≥ 10), la acción no. El agente abre una terminal interactiva con la API de Wazuh (:55000), el historial de `alerts.json`, `ss -tln` y `ps aux` precargados, y propone contención con el usuario presente. El bloque `<active-response>` de Wazuh está comentado: nada se mata ni se bloquea solo.
- **Microsegmentación:** los sensores cloud son solo de salida. El stack local de Wazuh sí escucha: seis puertos publicados, todos enlazados a `127.0.0.1`. Ninguno queda expuesto a la red — que no es lo mismo que "ceros puertos abiertos".
- **Controles DevSecOps:** ShellCheck, Gitleaks, Semgrep y Trivy (IaC) más dos controles funcionales. **No hay SCA** — nada escanea dependencias ni imágenes de contenedor.

2. Flujo de Telemetría y Gestión de Flota Centralizada



3. Matriz de Soporte EDR / XDR Corporativo

Sensor / Plataforma	Tipo de Paquete	Método de Despliegue en Omarchy	Comando CLI
CrowdStrike Falcon	.rpm (RHEL/SLES)	rpmextract + falconctl --cid (Modo eBPF)	omarchy-sec onboard falcon
Microsoft Defender (MDE)	.deb (Ubuntu)	debtap + MicrosoftDefenderATPOnboarding.py	omarchy-sec onboard defender
SentinelOne Singularity	.rpm / .deb	rpmextract + sentinelctl site-token	omarchy-sec onboard sentinelone
Palo Alto Cortex XDR	.sh Bundle	Script ejecutable + --distribution-token	omarchy-sec onboard cortex
Wazuh Open XDR/EDR	AUR / Nativo	paru -S wazuh-agent + agent-auth	omarchy-sec onboard wazuh

4. Mecanismo de Respuesta Autónoma con IA ("Call Agent")

A diferencia de los paneles tradicionales, que solo muestran un indicador estático, **Omarchy Sec** abre el agente de código de Omarchy con el contexto del incidente ya cargado. Importa distinguir qué es automático y qué no:

Flujo de Operación del Agente SOC:

1. **Disparo automático:** `omarchy-sec-watcher` sigue `alerts.json`. Con nivel ≥ 7 emite una notificación de escritorio; con nivel ≥ 10 además lanza el agente. Ambos umbrales son configurables por variable de entorno.
2. **Autenticación con la API REST (:55000):** obtiene un token bearer de `/security/user/authenticate` y lo cachea 10 minutos en `$XDG_RUNTIME_DIR` (directorio 0700, archivo 0600). Las credenciales las genera `setup.sh` al azar; no hay password por defecto.
3. **Volcado de telemetría:** `hostname`, sensor primario, la alerta disparadora, el resumen de `/agents`, las últimas 8 alertas de `alerts.json` (nivel, regla, descripción, timestamp, IP, archivo — *sin* campo de técnica MITRE), 12 líneas de `ss -tuLn` y 8 de `ps aux --sort=-%cpu`.
4. **Contención con el usuario presente:** la sesión es interactiva. El agente propone o ejecuta contención en esa terminal — `kill`, `ufw` — sujeta a tu confirmación y tu password de sudo. No hay respuesta activa de Wazuh configurada, y el proyecto **no** ejecuta reversiones con Btrfs/Snapper: recuperar un archivo manipulado es un paso manual.

5. Controles DevSecOps y Calidad Pre-PR

Corren dos cosas distintas: `tests/run_tests.sh` en local, a demanda, y `.github/workflows/security-ci.yml` en cada push y Pull Request a main. La tabla lista qué corre dónde y qué bloquea, no un resultado histórico.

Control	Herramienta	Dónde corre	¿Bloquea?
SAST — shell	ShellCheck	Local (bin/ + scripts/) y CI (bin/)	Sí
SAST — reglas genéricas	Semgrep (p/security-audit, p/secrets, p/bash)	Solo CI	Sí
Escaneo de secretos	Gitleaks	Local y CI	Sí
IaC & misconfiguración	Trivy config (HIGH, CRITICAL)	Local (--exit-code 1) y CI	Sí en local No en CI
Validación de plugin y manifiesto QML	Omarchy Plugin Validator	Solo local — el CLI de Omarchy no existe en un runner de GitHub	Sí

Control	Herramienta	Dónde corre	¿Bloquea?
Funcional: motor de detección	bin/omarchy-sec-detect	Solo local	Sí — verifica que el script corra, no que haya un sensor
Alcanzabilidad del dashboard	curl a https://localhost:9001	Solo local	Se saltea con el stack apagado

Límites de este pipeline, dichos explícitamente:

- **No hay SCA.** Nada escanea dependencias ni imágenes de contenedor, en ninguno de los dos pipelines, pese a que el nombre del workflow de CI lo menciona.
- **No se usa TruffleHog.** Solo corre Gitleaks; versiones anteriores de este informe listaban ambos.
- **La sonda del dashboard no es DAST.** Pregunta si el puerto responde. No hace fuzzing ni prueba la aplicación.
- **Los controles locales se saltean en silencio si la herramienta no está instalada,** y la corrida igual sale con código 0.

6. Repositorio Oficial y Comandos CLI

Repositorio en GitHub: <https://github.com/MAXI8594/omarchy-sec>

Instalación: `git clone https://github.com/MAXI8594/omarchy-sec && cd omarchy-sec && ./setup.sh`

`setup.sh` es el asistente completo: genera credenciales aleatorias, levanta el stack de Wazuh en Docker e invoca a `install.sh`. `install.sh` por sí solo instala únicamente los binarios CLI, el widget de barra y el servicio `systemd` de usuario — sin stack, no hay dashboard ni API que consultar.